Lab 11 - Charlie Caravati

In [61]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [53]:
```
def eval_composite_trap(M, a, b, f):
    """
    Composite Trapezoidal Rule approximation.
    Returns the integral approximation and None for the weights.
    """
    h = (b - a) / M
    x = np.linspace(a, b, M + 1)
    y = f(x)
    integral = (h / 2) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])
    return integral, None, None

def eval_composite_simpsons(M, a, b, f):
    """
    Composite Simpson's Rule approximation (requires M to be even).
    Returns the integral approximation and None for the weights.
    """
    if np.remainder(M,2)!=0:
        M = M-1
        message = "M must be even, calculating with M-1"
    else:
        message = None
    h = (b - a) / M
    x = np.linspace(a, b, M + 1)
    y = f(x)
    integral = (h / 3) * (y[0] + 4 * np.sum(y[1:-1:2]) + 2 * np.sum(y[2:-1:2]) + y[-1])
    return integral, message, None
```

In [11]:
```
def lgwt(N,a,b):
    """
     Computes the Legendre-Gauss nodes and weights on an interval
     [a,b] with truncation order N.

     Suppose you have a continuous function f(x) which is defined on [a,b]
     which you can evaluate at any x in [a,b]. Simply evaluate it at all of
     the values contained in the x vector to obtain a vector f. Then compute
     the definite integral using np.sum(f*w)

     Written by Greg von Winckel - 02/25/2004
     translated to Python - 10/30/2022
    """
    N = N-1
    N1 = N+1
    N2 = N+2
    eps = np.finfo(float).eps
    xu = np.linspace(-1,1,N1)

    # Initial guess
    y = np.cos((2*np.arange(0,N1)+1)*np.pi/(2*N+2))+(0.27/N1)*np.sin(np.pi*xu*N/N2)

    # Legendre-Gauss Vandermonde Matrix
    L = np.zeros((N1,N2))

    # Compute the zeros of the N+1 Legendre Polynomial
    # using the recursion relation and the Newton-Raphson method
    y0 = 2.
    one = np.ones((N1,))

    # Iterate until new points are uniformly within epsilon of old points
    while np.max(np.abs(y-y0)) > eps:

        L[:,0] = one
        L[:,1] = y
        for k in range(2,N1+1):
            L[:,k] = ((2*k-1)*y*L[:,k-1]-(k-1)*L[:,k-2])/k

        lp = N2*(L[:,N1-1]-y*L[:,N2-1])/(1-y**2)

        y0 = y
        y = y0-L[:,N2-1]/lp

    # Linear map from [-1,1] to [a,b]
    x = (a*(1-y)+b*(1+y))/2

    # Compute the weights
    w = (b-a)/((1-y**2)*lp**2)*(N2/N1)**2

    return x,w
```

In [12]:
```
def eval_gauss_quad(M,a,b,f):
  """
  Non-adaptive numerical integrator for \int_a^b f(x)w(x)dx
  Input:
    M - number of quadrature nodes
    a,b - interval [a,b]
    f - function to integrate

  Output:
    I_hat - approx integral
    x - quadrature nodes
    w - quadrature weights

  Currently uses Gauss-Legendre rule
  """
```

```
  x,w = lgwt(M,a,b)
  I_hat = np.sum(f(x)*w)
  return I_hat,x,w
```

In [57]:
```
def adaptive_quad(a,b,f,tol,M,method):
  """
  Adaptive numerical integrator for \int_a^b f(x)dx

  Input:
  a,b - interval [a,b]
  f - function to integrate
  tol - absolute accuracy goal
  M - number of quadrature nodes per bisected interval
  method - function handle for integrating on subinterval
         - eg) eval_gauss_quad, eval_composite_simpsons etc.

  Output: I - the approximate integral
          X - final adapted grid nodes
          nsplit - number of interval splits
  """
  # 1/2^50 ~ 1e-15
  maxit = 50
  left_p = np.zeros((maxit,))
  right_p = np.zeros((maxit,))
  s = np.zeros((maxit,1))
  left_p[0] = a; right_p[0] = b;
  # initial approx and grid
  s[0],x,_ = method(M,a,b,f);
  # save grid
  X = []
  X.append(x)
  j = 1;
  I = 0;
  nsplit = 1;
  while j < maxit:
    # get midpoint to split interval into left and right
    c = 0.5*(left_p[j-1]+right_p[j-1]);
    # compute integral on left and right spilt intervals
    s1,x,_ = method(M,left_p[j-1],c,f); X.append(x)
    s2,x,_ = method(M,c,right_p[j-1],f); X.append(x)
    if np.max(np.abs(s1+s2-s[j-1])) > tol:
      left_p[j] = left_p[j-1]
      right_p[j] = 0.5*(left_p[j-1]+right_p[j-1])
      s[j] = s1
      left_p[j-1] = 0.5*(left_p[j-1]+right_p[j-1])
      s[j-1] = s2
      j = j+1
      nsplit = nsplit+1
    else:
      I = I+s1+s2
      j = j-1
      if j == 0:
        j = maxit
  return I,x,nsplit
```

In [24]:
```
n = 5
a = 0.1
b = 2
eps = 10e-3
f = lambda x: np.sin(1/x)
```

In [58]:
```
adaptive_quad(a, b, f, eps, n, eval_composite_trap)
```

Out[58]:
```
(np.float64(1.1443198021525263), None, 5)
```

In [59]:
```
adaptive_quad(a, b, f, eps, n, eval_composite_simpsons)
```

Out[59]:
```
(np.float64(1.1458766725757545), 'M must be even, calculating with M-1', 5)
```

In [60]:
```
adaptive_quad(a, b, f, eps, n, eval_gauss_quad)
```

Out[60]:
```
(np.float64(1.1435746241385663),
 array([1.97771771, 1.89038646, 1.7625    , 1.63461354, 1.54728229]),
 3)
```

Composite trapezoid and composite simpsons arrive to the desired accuracy after 5 intervals. The Gaussian quadrature achieves the same accuracy after 3 intervals instead, which would indicate an advantage over the other two methods.

In [66]:
```
# specify the quadrature method
# (eval_gauss_quad, eval_composite_trap, eval_composite_simpsons)
method = eval_gauss_quad

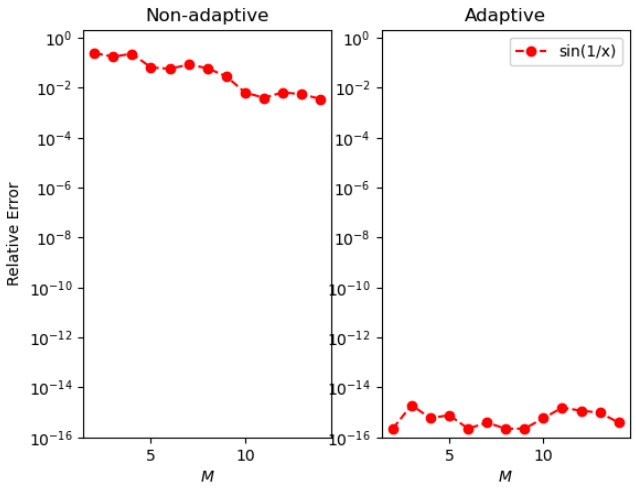
# interval of integration [a,b]
# function to integrate and true values
# TRYME: uncomment and comment to try different funcs
#        make sure to adjust I_true values if using different interval!
f = lambda x: np.sin(1/x);
I_true = 1.145580834099501
labl = "sin(1/x)"
#f = lambda x: 1./(np.power(x,(1./5.))); I_true = 5./4.; labl = '1/(x^{1/5})'
#f = lambda x: np.exp(np.cos(x)); I_true = 2.3415748417130531; labl = 'exp(cos(x))'
#f = lambda x: x**20; I_true = 1./21.; labl = 'x^{20}'
# below is for a=0.1, b = 2
#f = lambda x: np.sin(1./x); I_true = 1.1455808341; labl = 'sin(1/x)'


# number of nodes and weights, per subinterval
Ms = np.arange(2,15); nM = len(Ms)
# storage for error
err_old = np.zeros((nM,))
```

```
err_new = np.zeros((nM,))


# loop over quadrature orders
# compute integral with non adaptive and adaptive
# compute errors for both
for iM in range(nM):
  M = Ms[iM];
  # non adaptive routine
  # Note: the _,_ are dummy vars/Python convention
  # to store uneeded returns from the routine
  I_old,_,_ = method(M,a,b,f)
  # adaptive routine
  I_new,X,nsplit = adaptive_quad(a,b,f,eps,M,method)
  err_old[iM] = np.abs(I_old-I_true)/I_true
  err_new[iM] = np.abs(I_new-I_true)/I_true
  # clean the error for nice plots
  if err_old[iM] < eps:
    err_old[iM] = eps
  if err_new[iM] < eps:
    err_new[iM] = eps
  # save grids for M = 2
  if M == 2:
    mesh = X

# plot the old and new error for each f and M
fig,ax = plt.subplots(1,2)
ax[0].semilogy(Ms,err_old,'ro--')
ax[0].set_ylim([1e-16,2]);
ax[0].set_xlabel('$M$')
ax[0].set_title('Non-adaptive')
ax[0].set_ylabel('Relative Error');
ax[1].semilogy(Ms,err_new,'ro--',label=labl)
ax[1].set_ylim([1e-16,2]);
ax[1].set_xlabel('$M$')
ax[1].set_title('Adaptive')
ax[1].legend()
plt.show()
```



Adaptive methods are a clear improvement from the graph above!